

AN EXPLAINER

project-architect



An orchestrator that decides what to build
before Claude Code writes a single line.

11 PHASES · 6 SUBAGENTS · 19+ PROJECT TYPES · 16-CHECK QUALITY GATE

ALEXANDER FORD
ALEXANDER FORD VENTURES · MIT

GITHUB.COM/ALEXANDER-FORD-VENTURES/
PROJECT-ARCHITECT

A skill that interviews you before it builds anything.

project-architect is a Claude Code orchestrator skill. You install it, point it at an empty folder, and tell it what you want to build — in one sentence. From there it conducts a structured conversation across eleven phases, ending with a fully-committed repository: design docs, Architecture Decision Records, a per-folder CLAUDE.md router, a .claude/ tooling configuration, and three router slash commands (/scaffold, /implement, /iterate-design) wired and ready.

Not a generator. Not a template. An interview. The skill asks you to decide — with research backing each option — what your stack should be, what your interface should feel like, what your authentication model is, what your deployment target is, what your error budget looks like. Every meaningful decision is filed as an ADR. By the time anyone writes code, the project has already been thought through.

The shortest definition. project-architect makes the boring, important decisions visible — and then makes them portable, by writing them down in a form future-you and future-Claude can both read.

“Vibe coding” deserves a real architect.

There is a thing happening in software right now that the industry hasn't named well yet. People call it vibe coding — describing what you want in plain English and letting an AI write the code. It works astonishingly well for short tasks. It falls apart somewhere around hour three of a new project, when the model is confidently producing code that contradicts a decision it made an hour ago, in a file it has now forgotten exists.

The failure mode isn't intelligence. The failure mode is missing context. Models, like people, work better when they know what's been decided. The difference is that a person remembers decisions implicitly — the AI does not.

project-architect's bet is simple: front-load the decisions. Make them explicit. Write them down in a place the AI will re-read at the start of every future session. Then — and only then — start coding.

“The senior engineer's superpower isn't writing code faster. It's deciding which code not to write. project-architect is that decision conversation, captured.”

From code-first to decision-first.

The industry-standard, mature way to start a serious project — long before AI entered the room — looks like this: you write a one-page brief, you do a tech-stack RFC, you draft a CLAUDE.md, you file an ADR or two, you produce a scaffold plan, and only after all of that do you start typing into `main.ts`. Most teams skip these steps because they are tedious, undirected, and easy to put off. project-architect's job is to make them fast and pleasant by turning them into a guided conversation.

CODE - FIRST (THE TRAP)	DECISION - FIRST (THE ALTERNATIVE)
Open Claude. Paste a paragraph. Watch a file tree appear. Realise on day three you picked the wrong runtime, the wrong ORM, and the wrong auth story. Refactor. Re-paste. Repeat.	Open Claude. Run <code>/project-architect</code> . Answer eight multiple-choice questions about scope and constraints, six about the stack, four about architecture. Read the research notes the skill brought back. Approve the ADRs. Then let the scaffold land.
AI has no shared memory across sessions. Every restart re-derives everything from the current state of the code — which means earlier tradeoffs are invisible.	Every decision is written into <code>docs/decisions/</code> as an ADR. <code>CLAUDE.md</code> auto-loads at session start. Future-you and future-Claude both see why, not just what.
“Vibe coding doesn't scale.”	Vibe coding scales beautifully once you front-load the architecture. The vibe becomes iteration speed on a foundation that holds up.

Eleven phases, explained without jargon.

Each phase has a focused job. None of them are negotiable, but most of them are fast — typical bootstrap, from elevator pitch to locked v1.0 design, runs in about 90 minutes. Here is what happens, in human language:

PREFLIGHT

Checks the room: are you on Opus 4.7 with max effort? Are the recommended companion plugins installed? Is your project-architect copy current? Surfaces problems before they cost you an hour.

REPO INIT (OPTIONAL)

If the directory has no git yet, offers `git init` and optionally `gh repo create`. Skip if you already have a repo.

UNIVERSAL KICKOFF

Eight multi-choice questions. What type of thing is this? Who's the user? What's the time budget? Classifies your project into one of 19+ types so subsequent questions are relevant, not generic.

VISION & SCOPE

Type-specific drill-down. For a CLI tool: who runs it, on what OS, how do they discover commands? For a web app: auth model, multi-tenant or not, payments? A research-scout subagent fetches up-to-date context at the end of the phase.

TECH STACK

Real options, not opinions-from-2023. Per language, the skill picks the right CLI-UX library, the right web framework, the right database. Every major choice becomes an ADR. The skill runs cost research before recommending paid services.

COST MODELING

If your stack touches a metered service — Vercel, Supabase, AWS Bedrock, anything per-token — the skill estimates monthly cost at your expected scale. Filed into `COST_MODEL.md`.

ARCHITECTURE DEEP DIVE

Per-area drilling: how do auth tokens flow, what's the cache invalidation story, what's the failure-injection surface? Inline consistency-check so the architecture doesn't contradict the tech-stack choices.

DOCUMENT GENERATION

The big moment. document-author subagents run in parallel and write every design doc the project type requires. quality-gate-auditor then runs 16 cross-cutting checks (link integrity, ADR coverage, no placeholder text, consistent numbers, etc.). Findings auto-seed the next phase.

ITERATION

You read what's been written. The skill presents a menu of auditor findings plus open questions. Want to revisit the database choice? `decision-revisor` re-opens it, supersedes the old ADR, and updates every doc that referenced it.

POST-GENERATION SETUP (LOCK)

Final commit. Snapshot the docs to `docs/versions/v1.0/`. Set `state.locked = true`. From here on, design changes go through Phase 5 explicitly — no drift.

TOOLING EXECUTION + HANDOFF

Two specialised authors (`claude-md-author`, `claude-tooling-author`) consume the plan docs from Phase 4 and emit the final `CLAUDE.md` tree and `.claude/ tooling`. Then prints a restart message — future sessions auto-load the new `CLAUDE.md` as a router with `/scaffold`, `/implement <feature>`, `/iterate-design` available.

Why eleven phases and not five. Every phase exists because skipping it produced a measurable, recurring failure during real bootstraps. The quality-gate auditor exists because docs drifted from decisions. The LOCK phase exists because designs kept regressing. Phase 7 exists because `CLAUDE.md` kept getting written without a plan. Each phase pays for its own existence in problems it prevents.

The fastest path from idea to defensible architecture.

If you've been doing this for a while, you already know the value of an ADR, the pain of a stale `CLAUDE.md`, and the cost of discovering at deploy time that your framework can't do what you assumed. `project-architect`'s offer to you is straightforward:

- One conversation replaces what would otherwise be a week of solo RFC-writing. The skill remembers everything you decide and never asks twice.
- Every decision is an ADR. Sequential, supersession-aware, with a full audit trail. If you change your mind about the database in Phase 5, every doc that mentioned the old choice gets rewritten — automatically, with a new ADR filed superseding the old one.
- The 16-check quality gate runs after every doc-generation pass. It catches link rot, missing ADR references, broken JSON, drift in numbers across docs, ISO8601 timestamp violations, unresolved placeholders, and ten other things you'd otherwise discover three months in.
- Type-aware questioning. 19+ project types, each with its own decision tree. A CLI tool doesn't get asked about CDN caching; a SaaS doesn't get asked about `termios`. v2.3 added programming-language design as a first-class type, with 7 dedicated templates and 4 decision axes.
- Phase 7 hands off cleanly to `superpowers:writing-plans` for the actual implementation. The scaffold plan emitted by Phase 4 is exactly the input that `subagent-driven-development` wants. The boundary between design and code is sharp — and reversible.

The dirty secret: the people who already write ADRs by hand love this skill the most. It is not lowering the floor for them — it is raising the ceiling. The skill makes it cheap to file four ADRs when you would have filed one, because each one takes ninety seconds of clicking instead of fifteen minutes of writing.

If you can describe your idea, you can ship a real architecture.

Here is the thing the AI hype cycle keeps half-saying and never quite finishing: the bottleneck for non-developers building software has never been the code. It has been the decisions. Which framework? Which database? Which auth provider? Which hosting? Which payment processor? Each one has eight options, every option has tradeoffs, and there is no way to evaluate them without experience you don't have yet.

project-architect was designed so that someone who has never written production code can sit down, answer questions in plain language, and walk away with the same artifact a senior engineer would have produced — design docs, ADRs, a tech-stack rationale grounded in actual research, a cost estimate, and a scaffold plan ready to hand to Claude for implementation.

WHAT IT LOOKS LIKE FOR YOU, CONCRETELY

- Questions are multiple-choice, not free-form. You pick from real options the skill has researched. If you don't know what JWT means, the skill explains it inline before asking you to choose.
- Tradeoffs are surfaced, not hidden. “Option A is faster to ship and locks you into vendor X; Option B is slower to ship but portable.” You make the call; the skill doesn't pretend there's a right answer.
- Cost is estimated in dollars per month, at your expected scale, before you commit. No more discovering that the AI-powered features you assumed were “included” will cost \$1,200/month at 10,000 users.
- The end product is something you can show a developer. “Here is my design doc, here are my ADRs, here is my architecture.” You will be taken seriously, because the artifact is taken seriously. The format is the same one engineering teams use internally.
- If you decide to ship it yourself, the `/scaffold` and `/implement` commands wired up at the end give Claude Code the context it needs to write the actual code — chunk by chunk, on the foundation you already designed.

This is the real story of vibe coding. It is not “AI replaces engineers.” It is “the gap between knowing what you want and shipping it just collapsed” — but only for people who let the AI see the architecture first. project-architect is the bridge.

Artifacts, not promises.

At the end of a bootstrap, your repository contains the following — all committed, all readable, all consumable by future Claude sessions:

ARTIFACT	WHAT IT IS
CLAUDE.md + per-folder	A root router that loads into every Claude session, plus folder-level overlays where conventions differ. Future sessions wake up with full project context.
docs/PROJECT_OVERVIEW.md	The master hub. Vision, scope, decisions, links to every other doc. The first thing anyone — human or AI — should read.
docs/decisions/ (ADRs)	Every meaningful tradeoff. Sequentially numbered, supersession-tracked, dated. If a decision changes later, the chain shows the evolution.
docs/research/	Every research-scout finding the skill ran, archived. You can re-read the comparison that justified your runtime choice three months later.
docs/*_PLAN.md	Four design-first plan docs from Phase 4 (CLAUDE_MD, CLAUDE_TOOLING, SCAFFOLD, NEXT_STEP). Phase 7 consumes the first two; superpowers:writing-plans consumes the third; you consume the fourth.
docs/versions/v1.0/	Snapshot of every design doc at the moment of LOCK. If you regret something in v1.1, you can diff against this.
.claude/	Stack-aware tooling — settings.json, hooks for lint-on-save and test-on-stop, custom agents, slash commands. Tightened permissions when fewer-permission-prompts is installed.
.claude/commands/	Three router slash commands wired up: /scaffold (kick off implementation), /implement <feature> (add a feature on the existing architecture), /iterate-design (re-open the design loop without losing locked state).
docs/_architect_state.js on	The skill's own memory. Survives across sessions. Re-invocation enters wherever you left off.

Future possibilities.

v2.3 just added programming-language design as a first-class project sub-type — six PL flavours (general-purpose, DSL, query, configuration, educational, transpiler target) with seven dedicated design templates (grammar, semantics, type system, stdlib, toolchain, bootstrap plan, stability & RFC) and four decision axes calibrated to the 2026 state-of-the-art (LLVM 22.x, Cranelift, QBE, GraalVM/Truffle 24/25 LTS, Wasm 3.0, BEAM, dependent types as in Lean 4).

The roadmap below is not a roadmap so much as a horizon. project-architect is iteratively shipped — each release responds to lessons from real bootstraps. Here is what looks reachable from where we are now:

DIRECTION	WHY IT WOULD MATTER
PL-specific auditor checks (v2.4)	Grammar consistency, type-system soundness sketches, bootstrap-plan dependency cycles — quality-gate-auditor extended for the new programming-language type.
Polyglot stdlib design	Cross-language standard library design for project types that span multiple runtimes — a single source of truth for naming and semantics across Rust, Go, and TypeScript.
Markup-language sub-type	CommonMark variants, configuration markups, template languages — sibling to the programming-language sub-types, with its own template stack.
Live cost dashboards	COST_MODEL.md auto-syncs with the actual bills from your hosting providers, so the design-time estimate becomes a runtime delta you can act on.
Multi-project workspaces	Bootstrap a monorepo with three apps and shared packages — each gets its own design pass, all share a top-level architecture doc, consistency is auditor-enforced across packages.
Continuous architecture review	After lock, ongoing PRs are checked against the locked design — drift surfaces as a PR comment, with a one-click /iterate-design path to make the change deliberately.
The teaching mode	An optional “explain why this matters” toggle that, at every decision point, expands a sidebar with the engineering principles behind the question. project-architect as a learning instrument, not just a builder.

Three commands to start.

Open a terminal in a fresh directory. The whole bootstrap fits on a postcard:

```
claude plugin marketplace add alexander-ford-ventures/project-architect
claude plugin install project-architect@alexander-ford-ventures
claude
```

Then, inside Claude:

```
/effort max
/model      → Opus 4.7 (1M context)
/project-architect
```

Answer the questions. Read the research the skill brings back. Approve the ADRs. Read the docs that get generated. Iterate. Lock. Hand off to the scaffolder.

Ninety minutes from now, your repository has a foundation that takes most teams a week of meetings to produce. You will not have written a single line of production code — and that is the point. The code is the easy part now. The decisions that shape the code were always the hard part. project-architect makes them visible, portable, and cheap.

The deeper aspiration. If project-architect succeeds long-term, it is because it normalises something the industry already knows is true but rarely practises: designing before coding pays for itself many times over. AI didn't change that. AI made it cheaper. The window for “just start vibe-coding and figure it out” is real, but it ends at — conservatively — a thousand lines. Past that line, structure wins. Always has, always will.

GITHUB.COM/ALEXANDER-FORD-VENTURES/PROJECT-ARCHITECT · MIT LICENSED · v2.3.0 · MAY 2026

📦 SKILLFULLY MADE WITH PROJECT-ARCHITECT.